C. COMPUTAÇÃO
UNIPAC

CENTRO UNIVERSITÁRIO UNIPAC
SCIENTIA LIBERTAS ET FIDES
PRESIDENTE ANTÔNIO CARLOS

unipac.br
Barbacena

Computação Gráfica

Fundamentos e Pipeline Gráfico

Professor:

Dr. Rodrigo Fenandes dos Santos

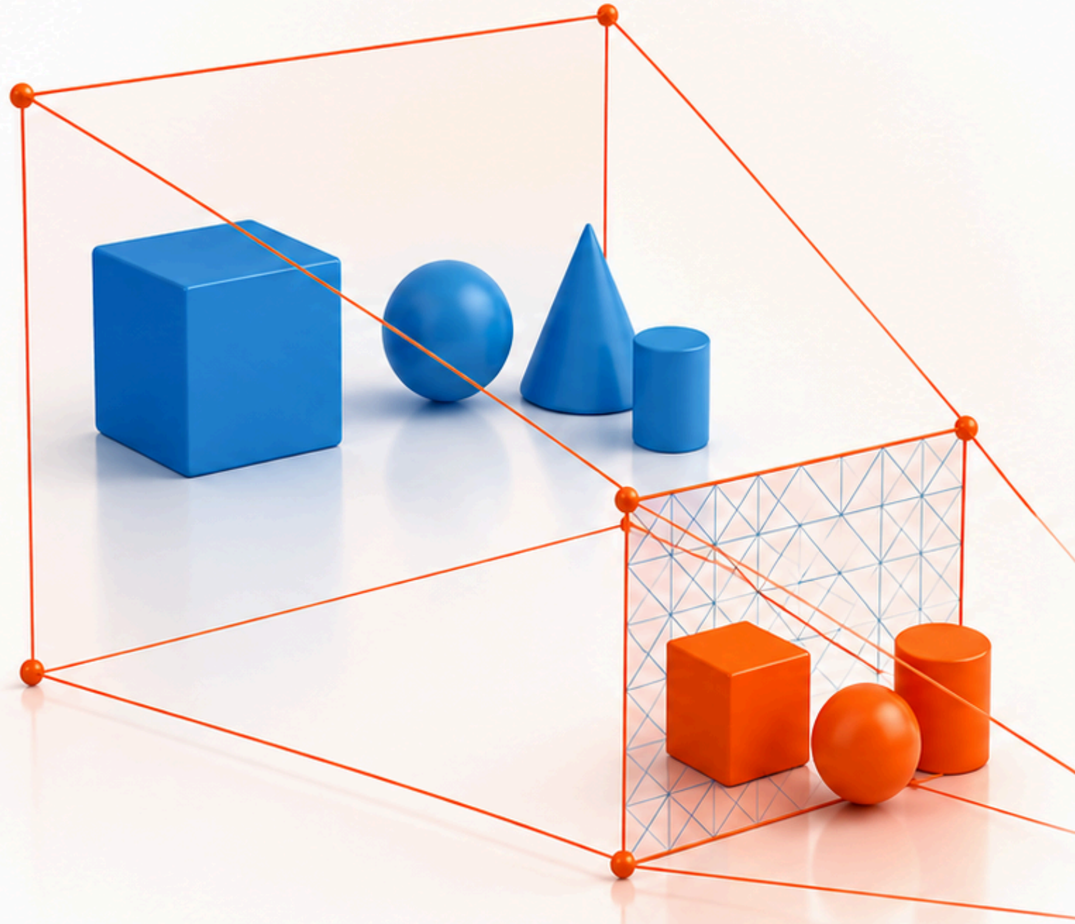
Aulas 05 e 06

unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

Como um mundo 3D se transforma em pixels na tela?

Da cena virtual ao raster: esta é a jornada que vamos desvendar.



OBJETIVOS DE APRENDIZAGEM

Ao final da aula, você será capaz de...

Da cena digital ao pixel final: compreenda o caminho completo.

01 Distinguir os elementos da imagem digital
Cena, raster, pixel e framebuffer.

02 Explicar quem faz o quê
CPU, GPU e shaders trabalhando em conjunto.

03 Percorrer o pipeline gráfico
Aplicação, geometria, rasterização e pixels.

04 Preparar o ambiente de experimentação
Python para os próximos exercícios práticos.

CENA 3D → PIPELINE → PIXELS
O mapa que orienta toda a disciplina

unipac.br
Barbacena

C. COMPUTAÇÃO UNIPAC

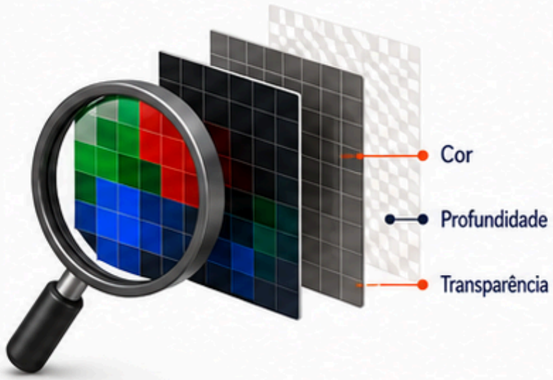
O percurso da aula

Da cena virtual à imagem exibida: quatro etapas para entender como a Computação Gráfica transforma dados em imagens.

1 Mundo visual

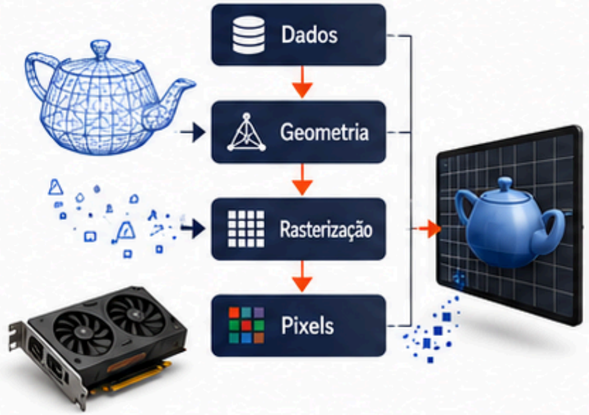
Partimos de cenas 2D e 3D: objetos, câmera, luz e textura definem o que será visto.

2 Pixels e buffers



Descobrimos como cor, profundidade e transparência são registradas antes de formar a imagem final.

3 Pipeline gráfico



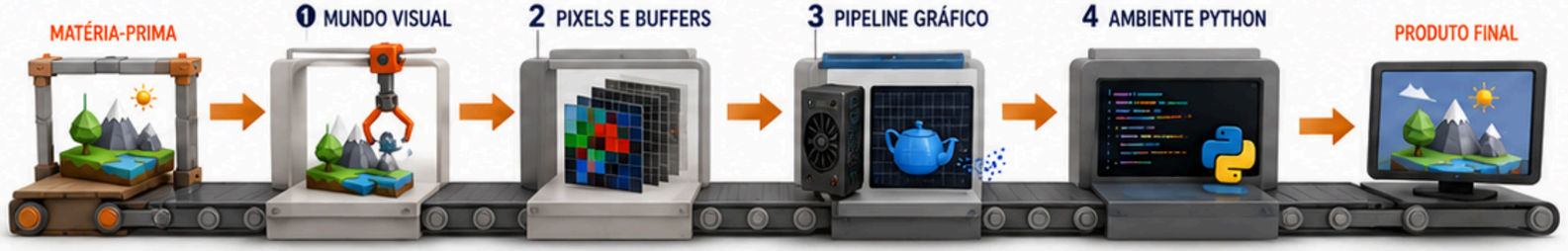
A GPU organiza os dados, processa a geometria, rasteriza a cena e gera fragments que se tornam pixels.

4 Ambiente Python

Preparamos Python, Pygame, NumPy e PyOpenGL para transformar os conceitos em programas gráficos.

ANALOGIA · Uma linha de produção visual

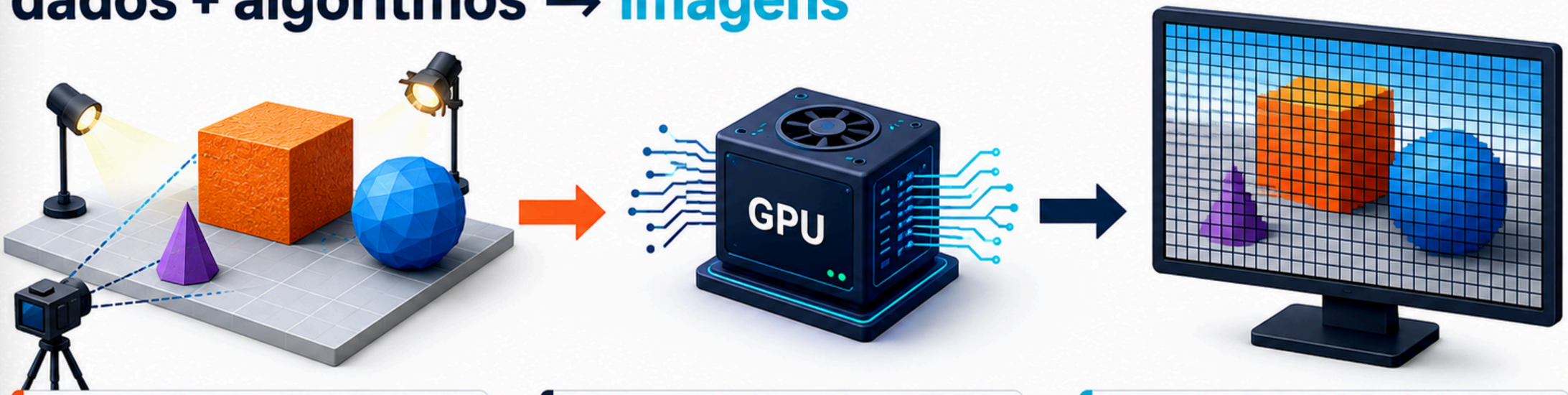
Como em uma fábrica, a cena entra como matéria-prima; cada estação transforma os dados até que o produto final apareça na tela: a imagem.



unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

Computação Gráfica:
dados + algoritmos → imagens



1. CENA
Objetos, luzes, texturas e câmera descrevem o mundo virtual.

2. PROCESSAMENTO
CPU e GPU aplicam cálculos que transformam dados em representação visual.

3. IMAGEM
O resultado é um raster de pixels exibido na tela.

ANALOGIA
Como uma receita: ingredientes + modo de preparo = prato final.

unipac.br
Barbacena

C. COMPUTAÇÃO UNIPAC

A Computação Gráfica está no seu cotidiano

Ela transforma dados, modelos e cenas digitais em imagens que ajudam a informar, projetar, tratar, entreter e interagir.

1 Visualização de dados

Gráficos e mapas revelam padrões e apoiam decisões.

Exemplo: painel de vendas por região

2 Medicina

Imagens médicas permitem observar estruturas internas do corpo.

Exemplo: tomografia 3D do crânio

3 Arquitetura

Modelos digitais simulam espaços antes da construção.

Exemplo: maquete 3D de uma residência

4 Entretenimento

Filmes e jogos criam cenários, personagens e efeitos visuais.

Exemplo: personagem 3D em um jogo

5 XR — realidade virtual e aumentada

Elementos digitais são vistos em ambientes reais ou imersivos.

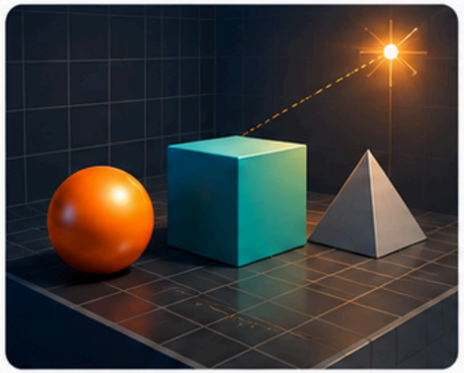
Exemplo: móvel virtual na sala pelo celular

unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

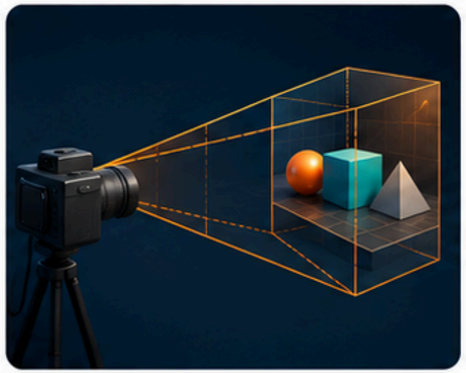
Renderizar: da cena 3D à imagem 2D

O processo que transforma dados de uma cena virtual nos pixels exibidos na tela.



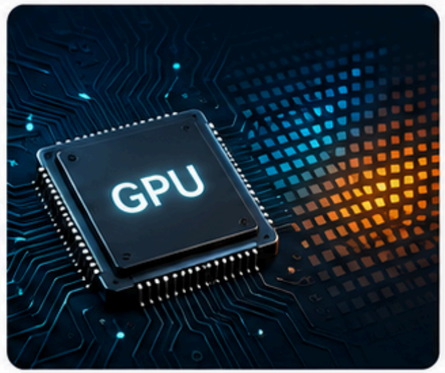
1. CENA 3D

Objetos, posições, materiais e luzes definem o mundo virtual.



2. CÂMERA

Define o ponto de vista e seleciona a parte da cena que será vista.



3. RENDERIZAÇÃO

A GPU calcula geometria, cores, iluminação e profundidade.

4. IMAGEM 2D

O resultado é um raster: uma matriz organizada de pixels.

IDEIA-CHAVE

A cena permanece tridimensional no computador; a imagem final é bidimensional e formada por pixels.

unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

Uma cena 3D combina elementos que precisam trabalhar juntos.

Cada componente contribui para o resultado que vemos na tela.

OBJETO
A geometria que define a forma 3D.

TEXTURA
A imagem aplicada à superfície do objeto.

LUZ
Define brilho, sombra e profundidade visual.

CÂMERA
Escolhe o enquadramento e o ponto de vista.

Câmera virtual e frustum

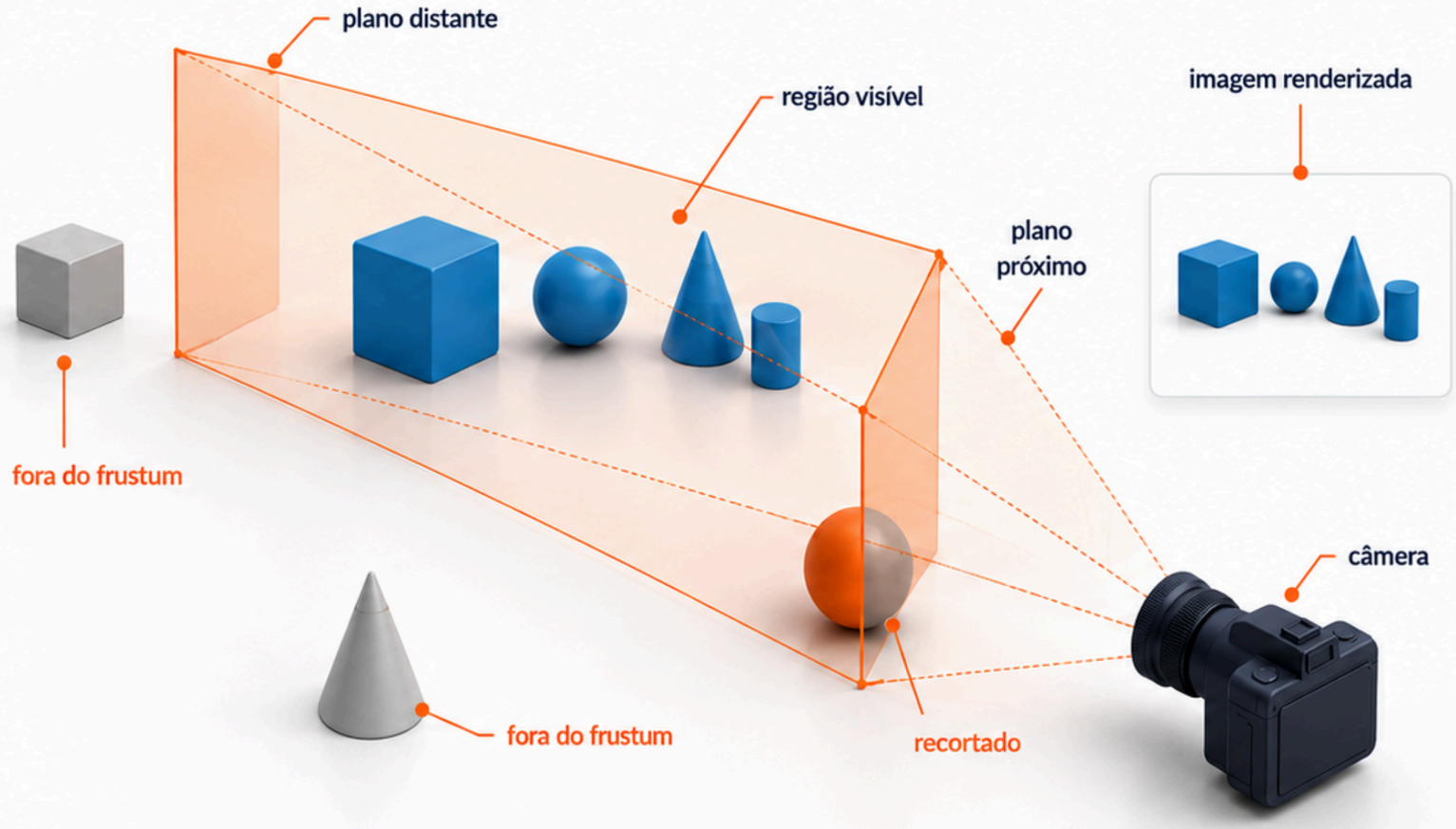
A câmera define o ponto de vista; o frustum define a região da cena que pode aparecer na tela.

1 POSIÇÃO E ORIENTAÇÃO
Definem onde a câmera está e para onde ela aponta.

2 FRUSTUM
Volume de visão que limita a parte da cena analisada pela câmera.

3 FORA DO VOLUME
Objetos fora do frustum são descartados; objetos cortados na borda são recortados.

FRUSTUM = CAMPO DE VISÃO DA CÂMERA



unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

Da imagem ao pixel

Uma imagem digital é formada por uma grade de pequenos pixels.

RASTER

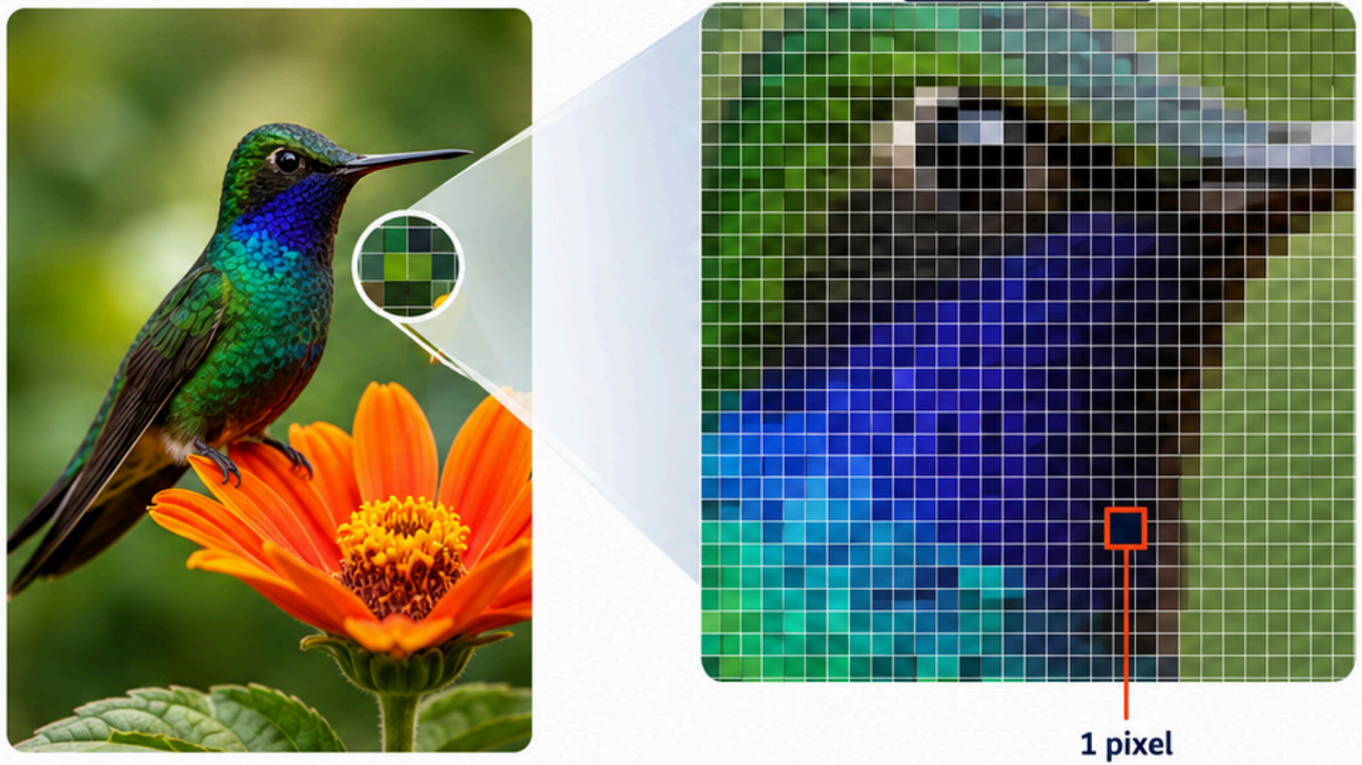
Matriz organizada em linhas e colunas que armazena os pixels da imagem.

PIXEL

Menor unidade visível da imagem. Cada pixel possui uma cor e uma posição.

RESOLUÇÃO

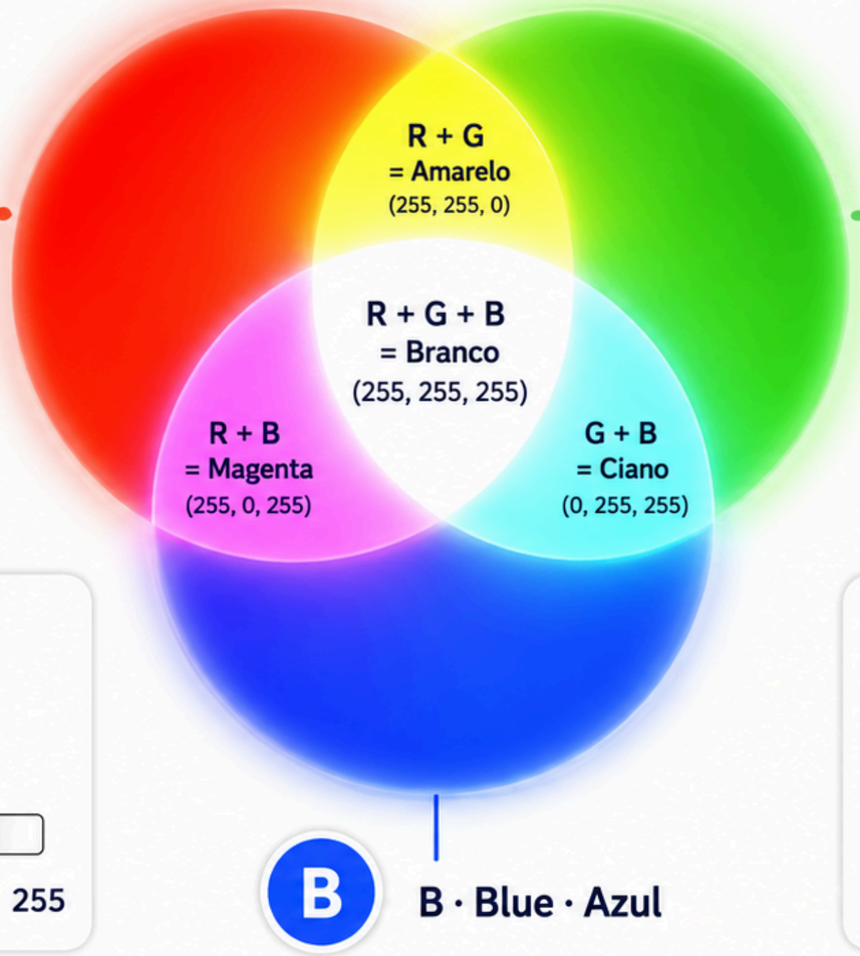
Quantidade de pixels: mais pixels permitem representar mais detalhes.



Ao ampliar demais, os detalhes viram quadrados de cor.

RGB: como as telas criam milhões de cores

RGB é um **modelo aditivo:** cada pixel combina intensidades de luz Vermelha (R), Verde (G) e Azul (B).



Um pixel da tela possui subpixels RGB.

Cada canal

Em telas de 8 bits, cada canal varia de 0 a 255.

0 — 255

Exemplos de pixels

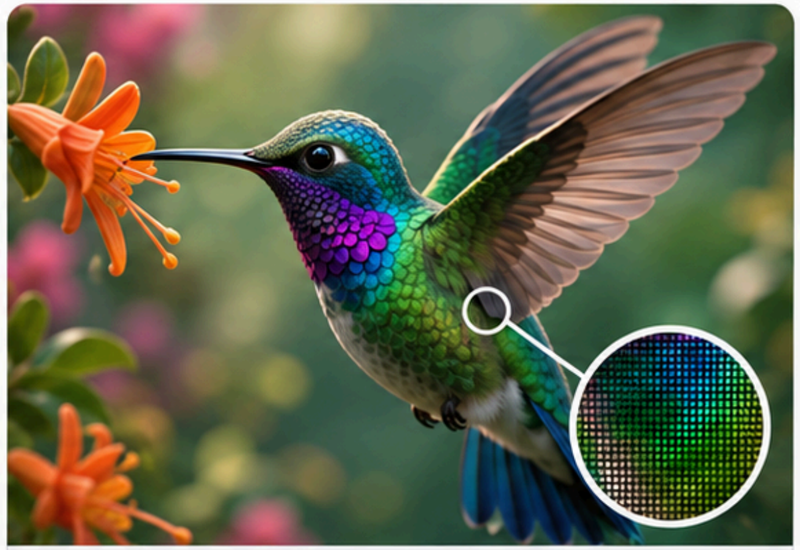
Preto	Vermelho	Amarelo	Branco
(0, 0, 0)	(255, 0, 0)	(255, 255, 0)	(255, 255, 255)

unipac.br
Barbacena

Qualidade de imagem: dois controles diferentes

Resolução define o nível de detalhe. Precisão define a variedade de cores.

Baixa resolução · 640 × 360



Alta resolução · 1920 × 1080

Analogia: resolução é a quantidade de azulejos; precisão é a quantidade de tintas disponíveis.

Precisão de cor

8 bits: transições visíveis

24 bits: transições suaves

Mais pixels ≠ mais cores. Uma imagem de qualidade precisa dos dois.

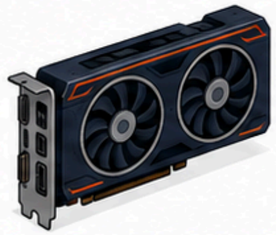
unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

Framebuffer: onde cada quadro é montado

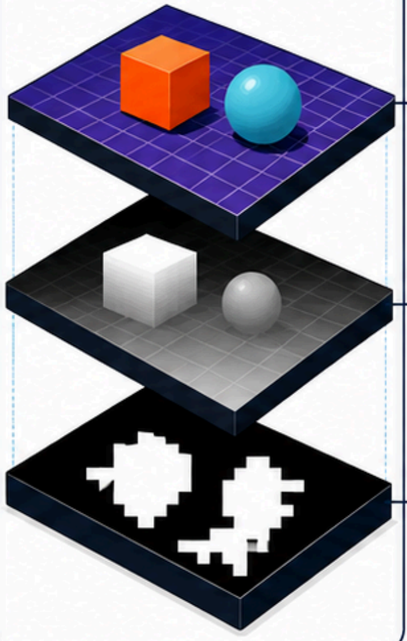
Área de memória temporária da GPU que guarda os resultados da renderização antes de enviá-los à tela.

GPU



RENDERIZA

FRAMEBUFFER



1 **COLOR BUFFER**
guarda a cor final de cada pixel

2 **DEPTH BUFFER**
guarda a distância e mantém o objeto mais próximo visível

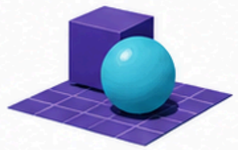
3 **STENCIL BUFFER**
marca regiões para efeitos e recortes

EXIBE

MONITOR

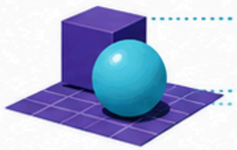
Em 3D, a **profundidade** evita que objetos distantes apareçam na frente.

SEM DEPTH BUFFER



Objeto distante (cubo) aparece na frente.

COM DEPTH BUFFER

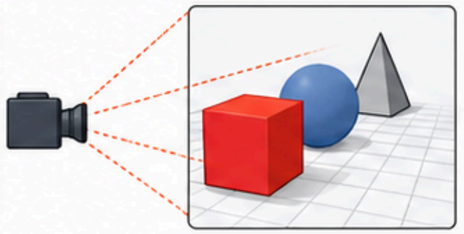


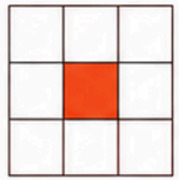
Objeto próximo (esfera) corretamente na frente.

A cada frame, os **buffers** são atualizados e a tela recebe uma **nova imagem**.

Três buffers, três decisões

O framebuffer combina três memórias de trabalho para construir a imagem final.

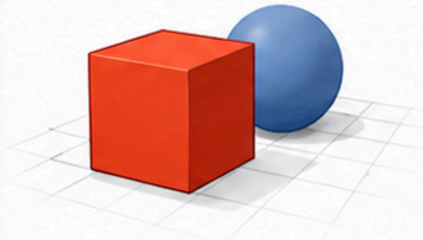




COLOR BUFFER

Qual cor o pixel terá?

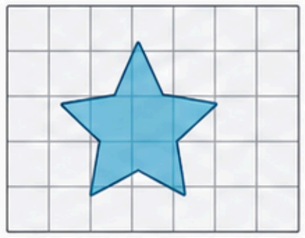
Guarda o valor RGB/RGBA que será exibido na tela.

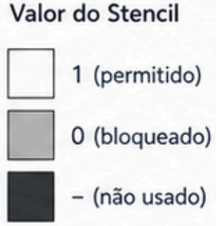


DEPTH BUFFER

Qual objeto está à frente?

Compara distâncias e mantém visível o objeto mais próximo da câmera.





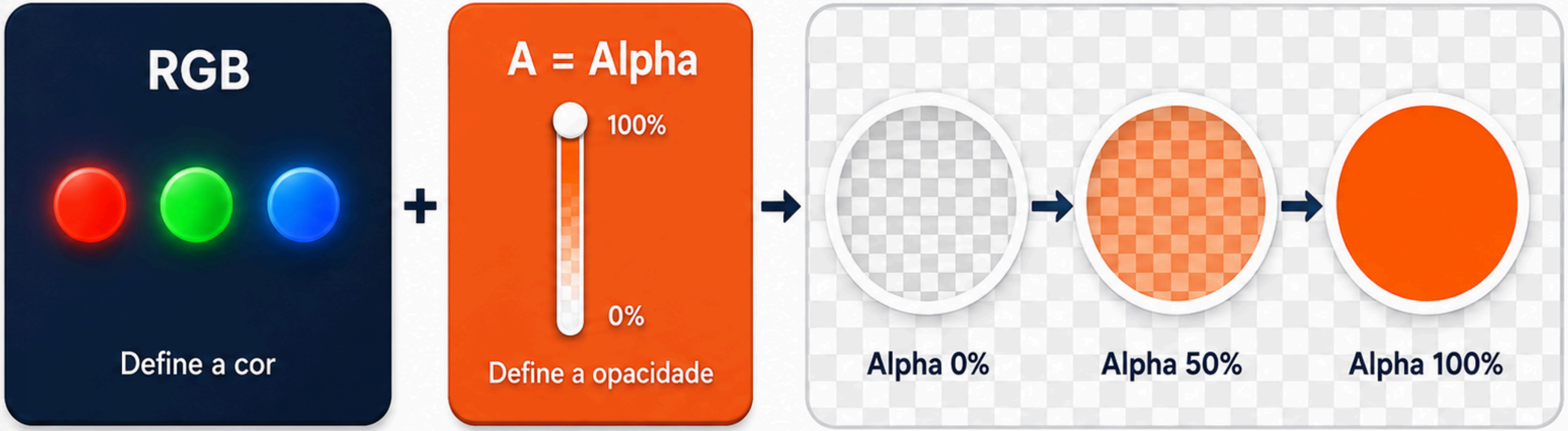
STENCIL BUFFER

Onde o efeito pode atuar?

Marca regiões da tela para aplicar recortes, espelhos e sombras.

RGBA: cor + transparência

Além de vermelho, verde e azul, o canal Alpha define quanto da cor aparece.



Exemplo: RGBA(255, 107, 0, 128) = laranja com 50% de transparência.

Em imagens com 8 bits por canal, Alpha varia de 0 a 255.

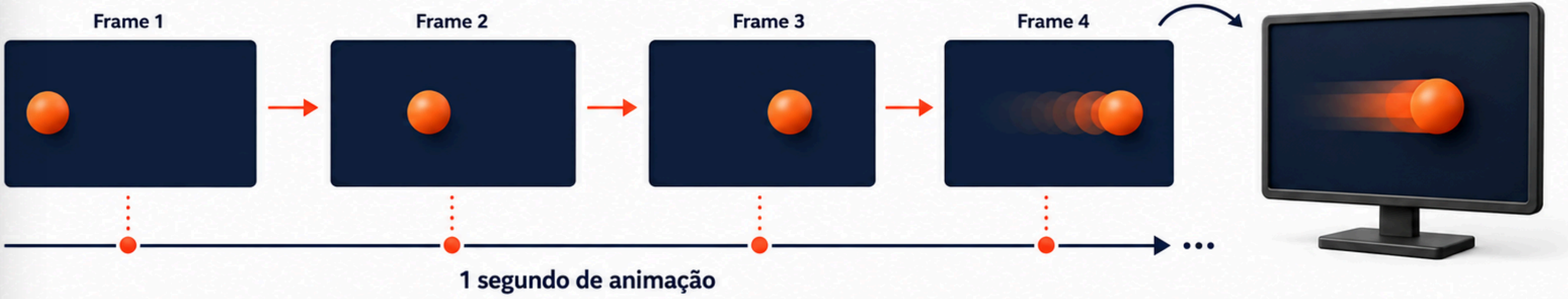
unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

Como uma animação cria movimento?

Animação é a exibição rápida de imagens estáticas chamadas frames.
Pequenas mudanças entre um frame e outro fazem o cérebro perceber movimento.

Exemplo: uma bola atravessando a tela



O que muda em cada frame?

A posição da bola é atualizada.
Ao exibir as imagens em sequência, vemos um deslocamento contínuo.

FPS = frames por segundo

24 FPS: cinema
60 FPS: jogos e monitores
90 FPS: realidade virtual

Efeito visual

Mais frames por segundo produzem movimentos mais fluidos; menos frames deixam o movimento mais perceptível entre uma posição e outra.

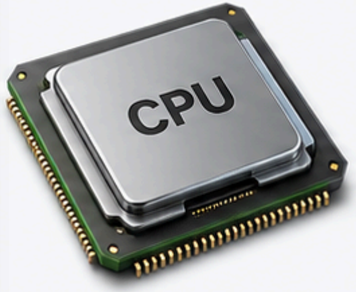
unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

CPU e GPU: papéis complementares no pipeline gráfico

A CPU organiza a cena; a GPU calcula milhares de elementos em paralelo.

CPU — coordena



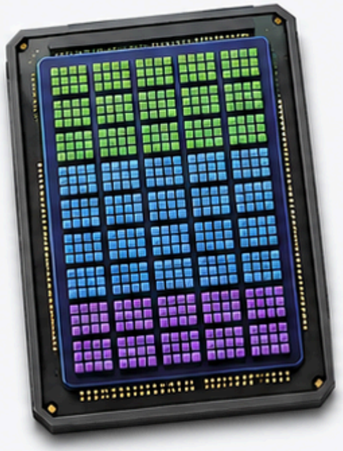
ENTRADAS E CENA

- Teclado
- Mouse
- Comandos
- Objetos
- Câmera
- Luzes
- ...

- Executa a lógica da aplicação
- Lê teclado, mouse e comandos
- Prepara objetos, câmera e dados
- Envia instruções para a GPU

dados da cena + comandos de desenho

GPU — processa em paralelo



VÉRTICES

PRIMITIVAS

FRAGMENTS

PIXELS

- Transforma milhares de vértices
- Executa shaders
- Rasteriza primitivas em fragments
- Calcula cor de muitos pixels ao mesmo tempo

SAÍDA FINAL

Imagem pronta para exibição

CPU: poucos núcleos complexos

Otimizados para tarefas variadas e sequenciais

GPU: muitos núcleos especializados

Otimizados para a mesma tarefa em paralelo

Exemplo: em um jogo, a CPU decide que a câmera se move; a GPU redesenha a cena para cada frame.

unipac.br
Barbacena

unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

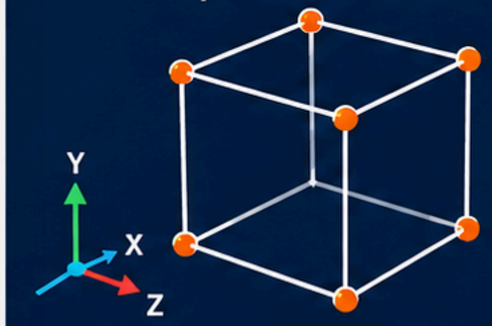
O que é um shader?

Um shader é um pequeno programa que a GPU executa para decidir como desenhar uma cena na tela.

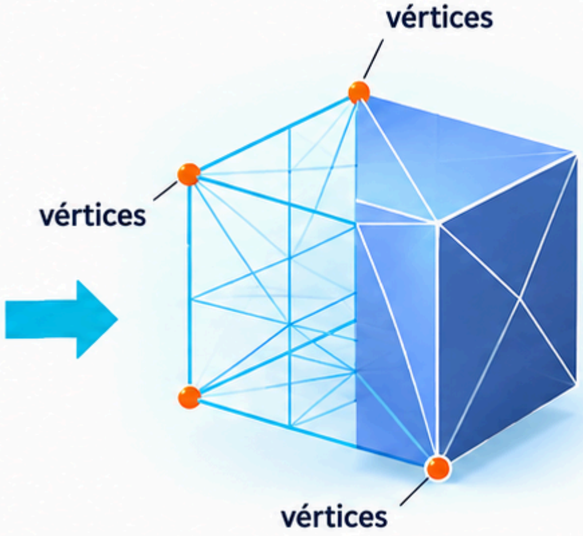
Exemplo: desenhando um cubo

1. Vertex Shader

Recebe os vértices e calcula onde cada ponto do cubo aparecerá na tela.



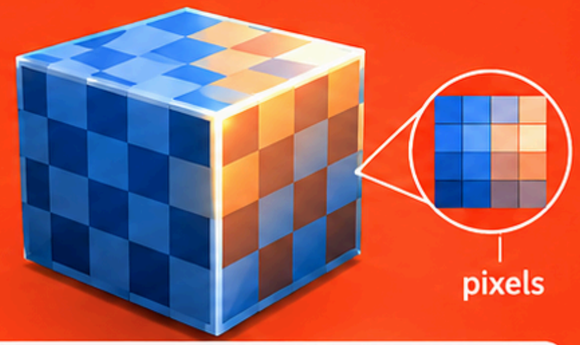
Move e projeta os pontos



A GPU forma os triângulos

2. Fragment Shader

Para cada fragmento visível, calcula a cor final usando textura, luz e sombra.

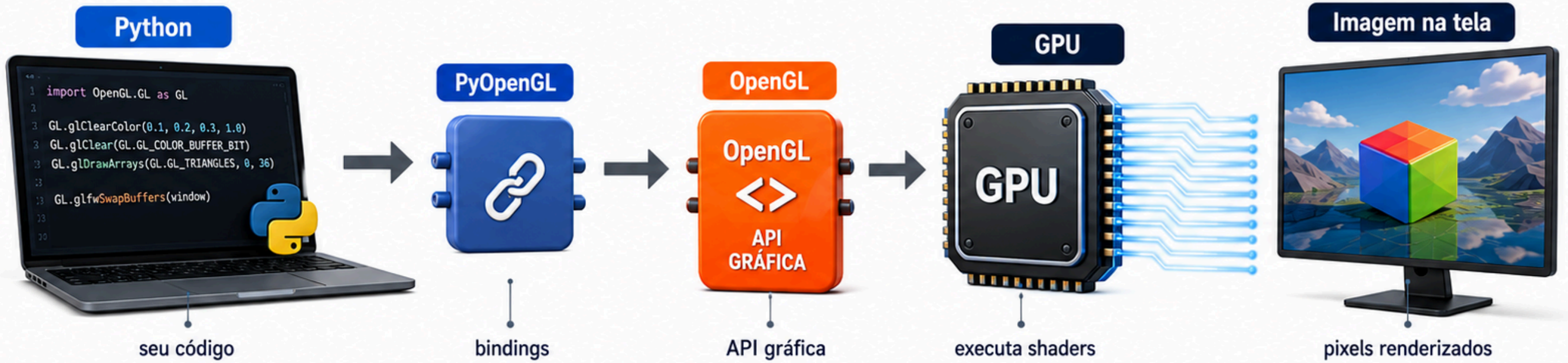


Define a aparência de cada pixel

Em resumo: o vertex shader define a forma e a posição; o fragment shader define a cor e o efeito visual.

APIs: como a aplicação conversa com a GPU

Uma API gráfica fornece comandos para enviar dados, configurar o processamento e solicitar a geração da imagem.



Principais APIs gráficas

DirectX
- Windows e Xbox
- Shaders: HLSL

Metal
- Apple
- Shaders: Metal Shading Language

OpenGL
- Multiplataforma
- Shaders: GLSL

No nosso curso

Escrevemos em Python. O PyOpenGL traduz nossas chamadas para a API OpenGL; nela, os shaders em GLSL orientam a GPU a produzir a imagem.

Foco da disciplina: OpenGL + GLSL

unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

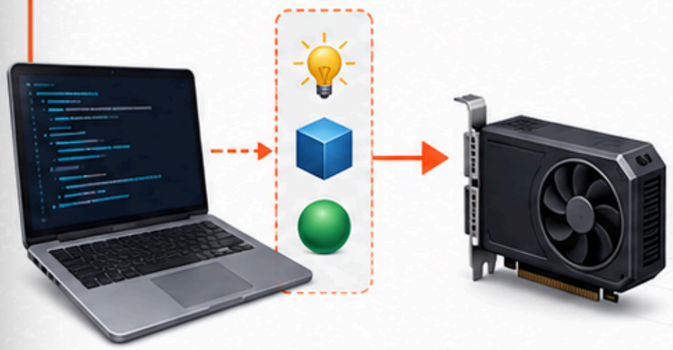
Pipeline gráfico: da cena 3D aos pixels

Cada etapa transforma a cena até formar a imagem exibida na tela.

1

1. Aplicação

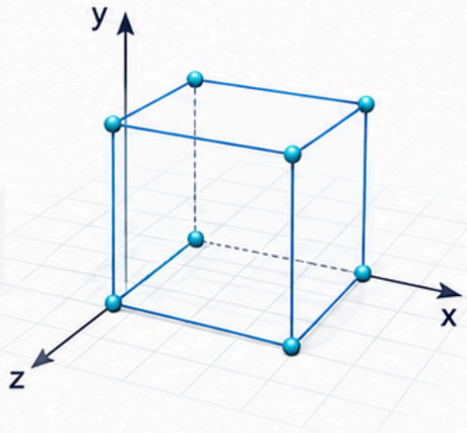
A CPU organiza a cena: objetos, câmera, luzes e comandos de desenho.



2

2. Geometria

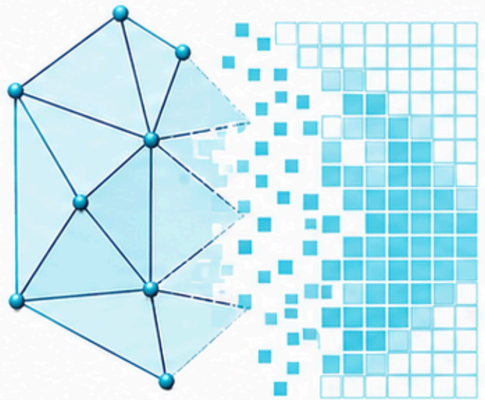
O vertex shader posiciona os vértices e monta as formas no espaço 3D.



3

3. Rasterização

Triângulos da malha são convertidos em muitos fragments, candidatos a pixels.



4

4. Pixels

O fragment shader calcula cor, luz e textura para cada fragment visível.

Dados da cena → geometria processada → fragments → pixels coloridos

Pipeline gráfico: uma linha de produção visual

Cada etapa transforma dados em uma imagem que vemos na tela.

1. Dados da cena

Vértices, cores, texturas e posições dos objetos.

2. Geometria

A GPU transforma e organiza formas 3D a partir desses dados.

3. Rasterização

As formas são convertidas em fragments: candidatos a pixels.

4. Imagem final

Os fragments recebem cor e formam a imagem exibida na tela.

dados da cena → formas 3D → fragments → pixels na tela

Etapa 1 — A aplicação prepara a cena

Antes de a GPU renderizar, o programa organiza tudo o que será desenhado.

1 Janela e loop

Cria a janela e atualiza a cena a cada frame.

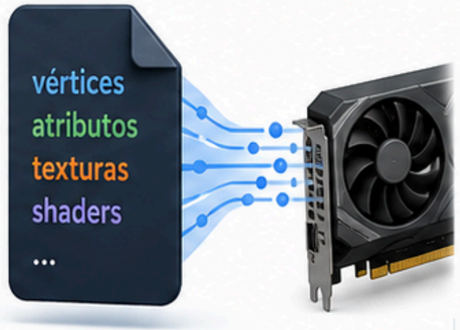
2 Entrada e lógica

Lê comandos do usuário e calcula movimento, física e colisões.

3 Dados da cena

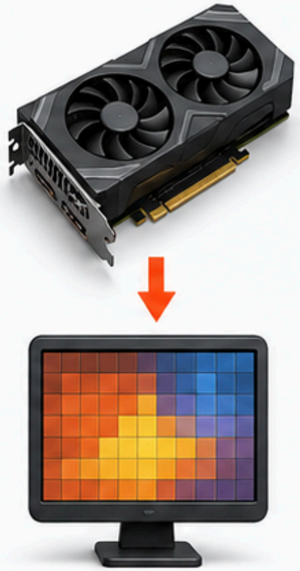
Organiza objetos, posições, cores, texturas e luzes.

4 Envio para a GPU



Transfere vértices, atributos, texturas e shaders para o processamento gráfico.

GPU



A GPU recebe os dados prontos para renderizar.

Exemplo: em um jogo

Ao pressionar **W**, a aplicação move o personagem, atualiza a cena e envia os novos dados à GPU.

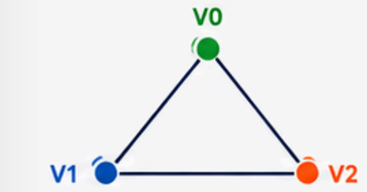
unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

VBOs e VAOs: como os vértices chegam à GPU

Antes de desenhar um objeto, o programa organiza os dados que a GPU precisa ler.

Dados dos vértices



Vértice	posição (x, y, z)	cor (r, g, b)	normal (nx, ny, nz)	uv (u, v)
V0	0.0, 1.0, 0.0	1.0, 0.0, 0.0	0.0, 0.0, 1.0	0.5, 1.0
V1	-1.0, -1.0, 0.0	0.0, 1.0, 0.0	0.0, 0.0, 1.0	0.0, 0.0
V2	1.0, -1.0, 0.0	0.0, 0.0, 1.0	0.0, 0.0, 1.0	1.0, 0.0

VBO

memória da GPU

0.0	1.0	0.0	1.0	0.0	0.0
-1.0	-1.0	0.0	0.0	1.0	0.0
1.0	-1.0	0.0	0.0	0.0	1.0
...	...	...	...	...	...

dados dos vértices armazenados na GPU

VAO

configuração de atributos

Atributo	VBO	Tamanho	Tipo	Offset	Stride	layout
posição: (x, y, z)	0	3	FLOAT	0	32	layout(location = 0)
cor: (r, g, b)	0	3	FLOAT	12	32	layout(location = 1)
normal: (nx, ny, nz)	0	3	FLOAT	24	32	layout(location = 2)
uv: (u, v)	0	2	FLOAT	36	32	layout(location = 3)

O VAO vincula o VBO e descreve como cada atributo está organizado na memória.

Vertex Shader

O shader lê os atributos conforme a configuração do VAO e processa os vértices.

Triângulo na tela

VBO — Vertex Buffer Object

Armazena na memória da GPU os dados do objeto: posições, cores, normais e coordenadas de textura.

VAO — Vertex Array Object

Guarda a configuração: informa ao shader onde cada atributo está e como ele deve ser interpretado.

Exemplo prático

Para desenhar um triângulo, o VBO envia os valores dos 3 vértices; o VAO explica que cada grupo de 3 números representa uma posição (x, y, z).

- posição: (x, y, z)
- cor: (r, g, b)
- normal: (nx, ny, nz)
- uv: (u, v)

```
layout(location = 0)
(ex.: posição)
```

VBO = dados armazenados | VAO = instruções para ler os dados

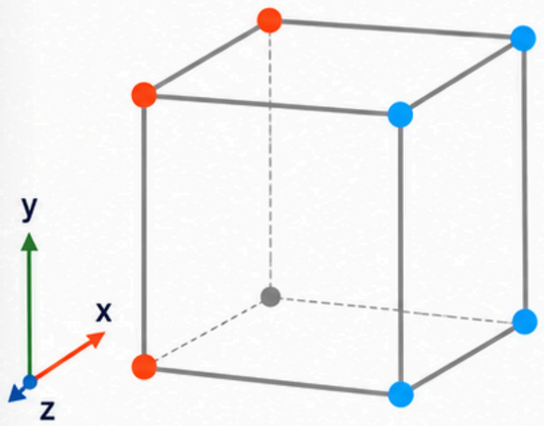
unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

ETAPA 2 — PROCESSAMENTO DE GEOMETRIA

O vertex shader calcula a posição final de cada vértice da malha.

1. Malha original



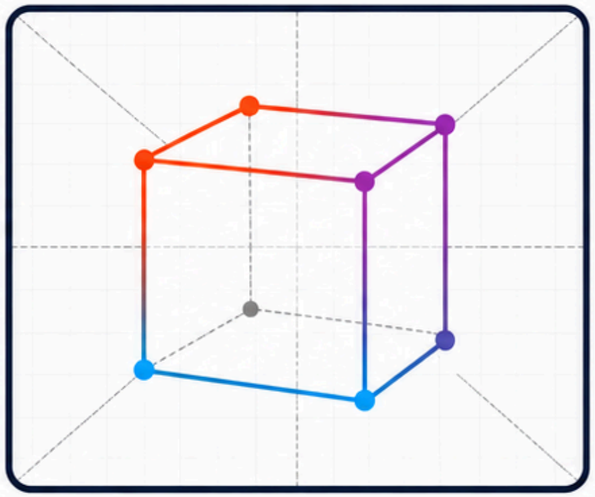
2. Vertex shader

vertex shader

```
// GLSL (conceitual)
vec4 posicaoFinal;
// Transformações do espaço 3D
posicaoFinal = Projecao ×
Visao × Modelo × vertice;
```

O vertex shader transforma a posição de cada vértice do espaço do objeto para o espaço de tela (coordenadas de recorte).

3. Posição final na tela



O que entra?
Posições e atributos de cada vértice.

O que sai?
Coordenadas prontas para formar a imagem.

Exemplo: ao girar um cubo, o vertex shader recalcula a posição de seus 8 vértices a cada *frame*.

unipac.br
Barbacena

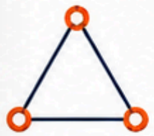
C. COMPUTAÇÃO
UNIPAC

Malhas: a estrutura dos objetos 3D

Uma malha organiza vértices, arestas e triângulos para formar a superfície de um objeto.

Vértice

Um ponto com posição no espaço e atributos, como cor, normal e coordenada UV.

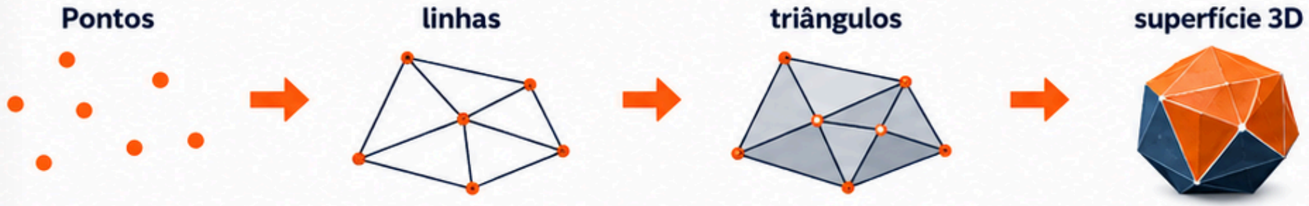


Aresta e face

As arestas conectam vértices; três arestas formam uma face triangular.

Primitivas

A GPU recebe pontos, linhas ou triângulos. Os triângulos são a base mais comum das malhas 3D.



Exemplo: um cubo

Mesmo um cubo é desenhado com 12 triângulos. Objetos mais detalhados usam milhares ou milhões deles.

unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

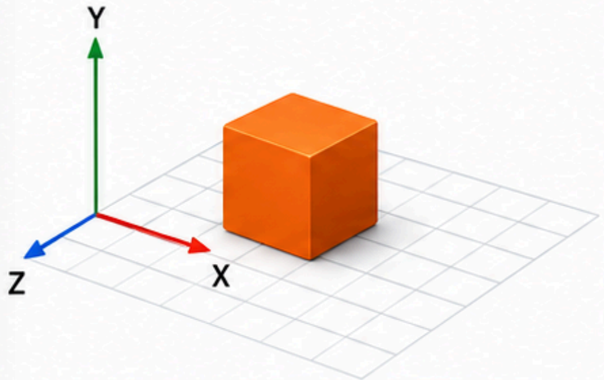
Três transformações posicionam a cena.

O mesmo cubo muda de referência em cada etapa do pipeline.

1 · MODELO

Do objeto para o mundo

Define posição, rotação e escala do objeto na cena.

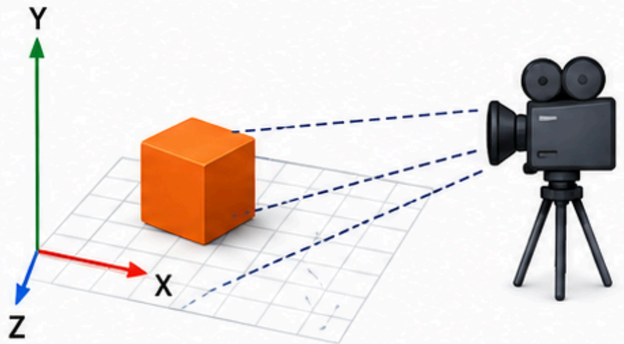


espaço local → mundo

2 · VISUALIZAÇÃO

Do mundo para a câmera

Reposiciona a cena conforme o ponto de vista da câmera.

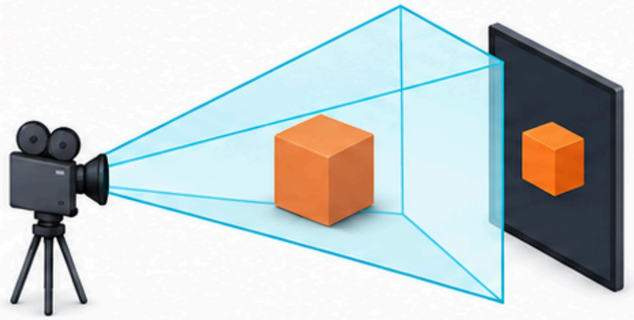


mundo → espaço da câmera

3 · PROJEÇÃO

Da câmera para a tela

Converte a visão 3D em coordenadas prontas para recorte e exibição.



câmera → clip space

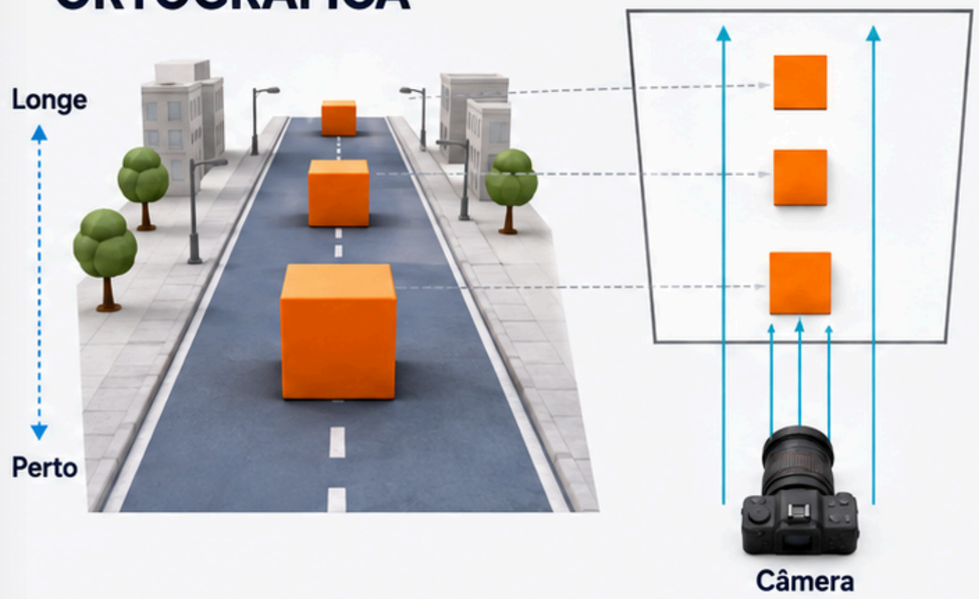
Ideia-chave: o objeto não muda de forma; muda o sistema de coordenadas usado para representá-lo.

unipac.br
Barbacena

Projeção ortográfica × perspectiva

Duas formas de levar uma cena 3D para a tela

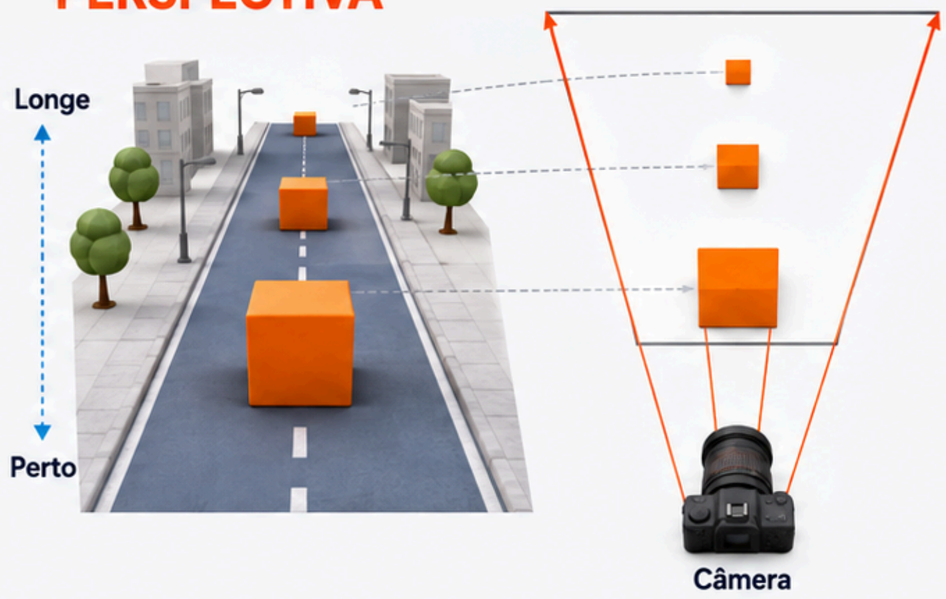
ORTOGRÁFICA



Objetos mantêm o mesmo tamanho aparente.

Exemplo: mapas, CAD e plantas técnicas

PERSPECTIVA



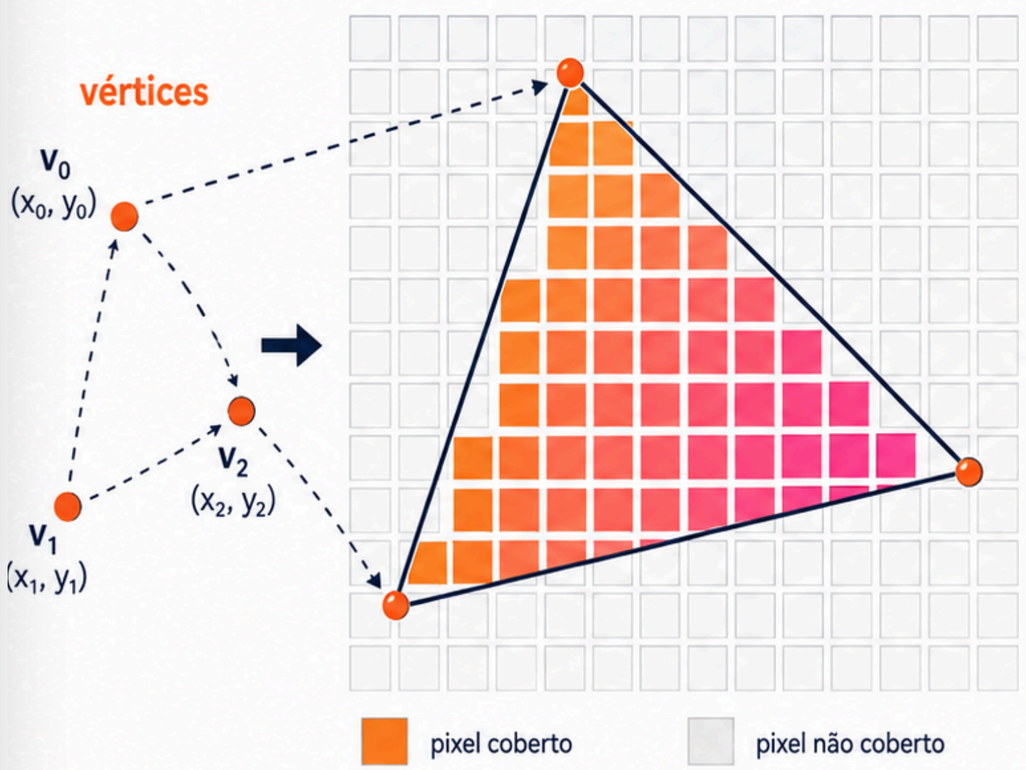
Objetos distantes parecem menores.

Exemplo: jogos, filmes e realidade virtual

A diferença está na projeção: distância altera a escala apenas na perspectiva.

Rasterização: da forma geométrica aos fragments

Nesta etapa, a GPU descobre quais pixels da tela são cobertos por cada primitiva.



1. Primitiva

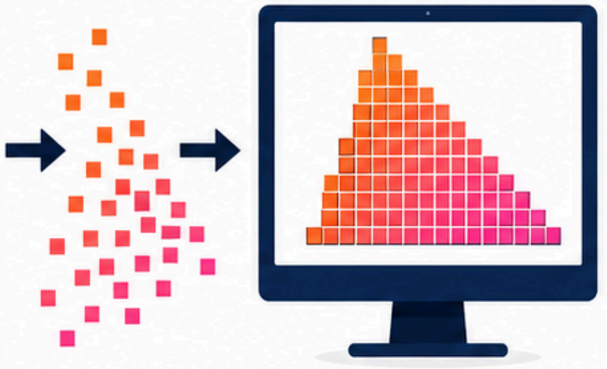
Vértices formam uma linha ou um triângulo.

2. Cobertura

A grade é testada: quais pixels ficam dentro da forma?

3. Fragment

Cada pixel coberto gera um fragment com dados para a próxima etapa.



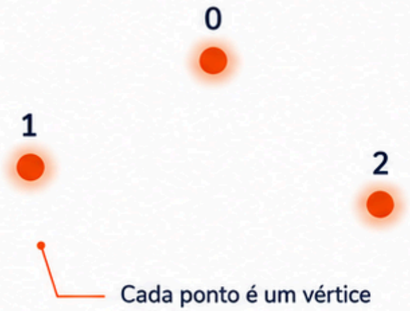
EXEMPLO | Um triângulo pode cobrir centenas de pixels — cada um deles se torna um fragment.

Primitive Assembly: dos vértices às formas

A GPU recebe vértices e os agrupa em primitivas, as formas básicas que serão rasterizadas.

1 1. PONTOS

Cada vértice aparece sozinho.

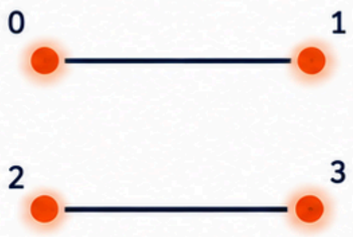


Cada ponto é um vértice individual.

`GL_POINTS`

2 2. LINHAS

Cada par de vértices forma um segmento.

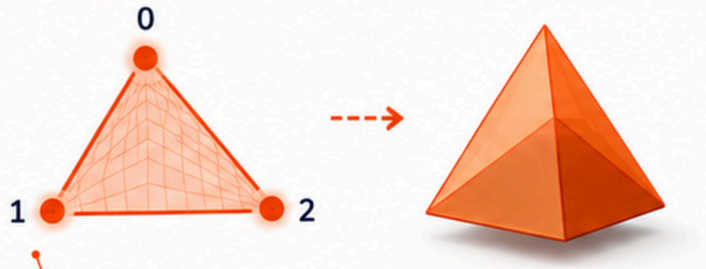


Vértices são agrupados de 2 em 2 para formar segmentos.

`GL_LINES`

3 3. TRIÂNGULOS

Cada grupo de 3 vértices cria uma face.

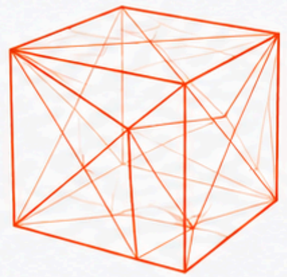


Três vértices definem uma face triangular.

`GL_TRIANGLES`

Por que triângulos?

Eles são sempre planos, simples de processar e permitem construir qualquer malha 3D.

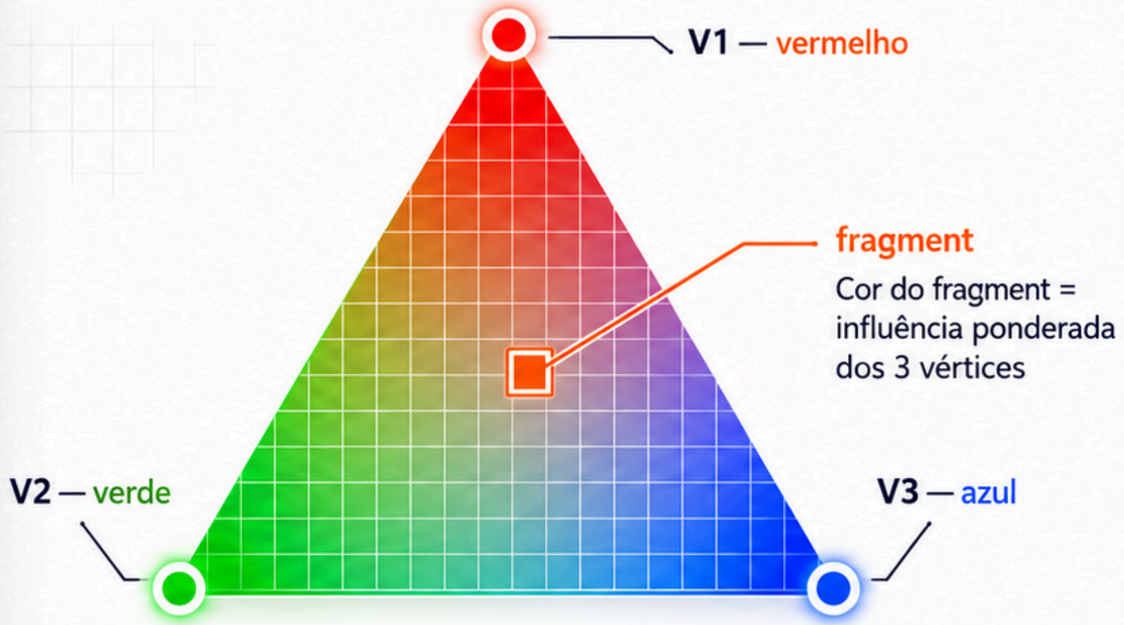


unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

Interpolação: transições suaves entre os vértices

Na rasterização, cada fragment recebe valores calculados a partir dos vértices próximos.



O que muda?

Cor, coordenadas de textura e normais variam gradualmente pela superfície.

Como é calculado?

Vértices mais próximos exercem maior influência no valor final.

Por que importa?

Evita mudanças bruscas e deixa cores, iluminação e texturas contínuas.

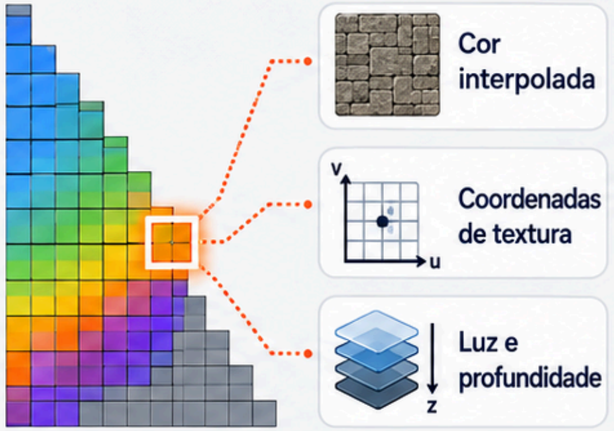
Exemplo prático

Com três vértices coloridos, os fragments entre eles recebem cores intermediárias — por isso não vemos faixas separadas.

Processamento de pixels: a cor final de cada fragment

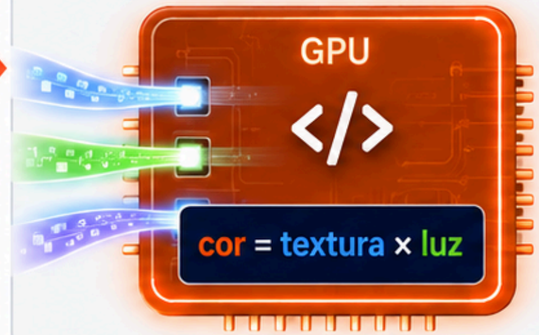
Depois da rasterização, cada fragment recebe dados e o fragment shader calcula sua aparência.

1. Fragment recebe dados



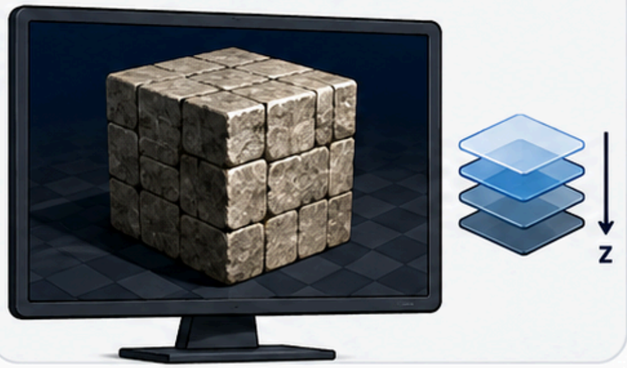
2. Fragment shader calcula

Um pequeno programa da GPU combina esses dados para definir RGB e transparência (Alpha).



3. Cor final no framebuffer

O resultado passa pelos testes de profundidade e transparência antes de aparecer na tela.



EXEMPLO: UM BLOCO DE PEDRA ILUMINADO

A textura fornece o padrão; a luz deixa a face voltada para a lâmpada mais clara e a face oposta mais escura.

Fragment = candidato a pixel • **Fragment shader = programa que define sua aparência**

unipac.br
Barbacena

C. COMPUTAÇÃO UNIPAC

Texturas, iluminação, profundidade e transparência refinam a imagem

1 TEXTURA

Aplica uma imagem à superfície e acrescenta detalhes visuais.

Exemplo: madeira, tijolo ou metal em um cubo 3D.

2 ILUMINAÇÃO

Calcula como a luz incide no objeto, criando brilho e sombras.

Exemplo: uma lanterna ilumina apenas um lado da cena.

3 DEPTH TEST

Compara distâncias e exibe somente o fragmento mais próximo da câmera.

Exemplo: o cubo da frente oculta o objeto que está atrás.

4 TRANSPARÊNCIA

Combina a cor do objeto com o fundo por meio do canal Alpha.

Exemplo: vidro azul permite enxergar o cubo ao fundo.

RESULTADO: uma cena 3D com materiais, luz, visibilidade e transparência coerentes.

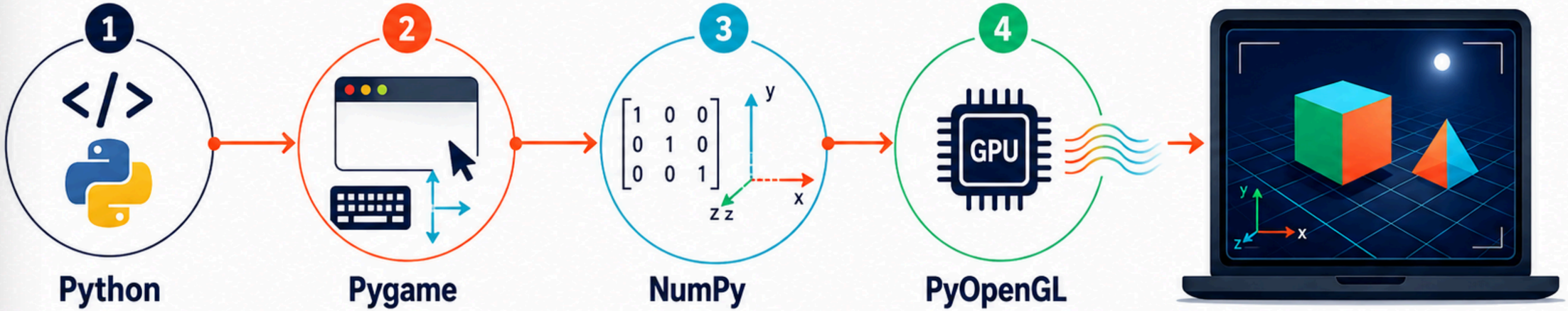
TEXTURA + ILUMINAÇÃO + DEPTH TEST + TRANSPARÊNCIA =

unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

Na próxima aula, vamos preparar o ambiente.

DO CÓDIGO À PRIMEIRA CENA 3D



Python
Escreve a lógica da aplicação e organiza o projeto.

Pygame
Cria a janela e captura teclado e mouse.

NumPy
Trabalha vetores, matrizes e transformações.

PyOpenGL
Envia comandos do Python para a GPU com OpenGL.

Resultado: você terá um ambiente pronto para criar e visualizar suas primeiras cenas gráficas.

Próximo passo:
instalação e teste

CONTATO

rodrigofernandes@unipac.br

OBRIGADO!!